Задание 1. Анализ повседневного приложения

Мессенджер Telegram — гибридную систему, использующую несколько протоколов.

На транспортном уровне применяется TCP как основной канал связи с серверами. Поверх него работает собственный протокол MTProto, обеспечивающий шифрование и мультиплексирование. По этому каналу идут текстовые сообщения, история чатов, синхронизация и файлы.

UDP используется для голосовых и видеозвонков (стек WebRTC, медиапоток RTP/SRTP), где важнее минимальная задержка, чем гарантированная доставка.

На прикладном уровне задействованы HTTPS (веб-версия, API, боты) и WebSocket (real-time обновления в браузерном клиенте, так как браузер не открывает raw TCP-сокет).

Задание 2. Обоснование стека и альтернативы

Выбор обусловлен характером трафика. Сообщения и файлы критичны к потере и порядку — поэтому TCP с гарантированной упорядоченной доставкой. Звонки критичны к задержке — потеря пакета лучше, чем ожидание повторной передачи, поэтому UDP с собственными механизмами восстановления (FEC, jitter buffer). WebSocket нужен, потому что браузер не может открыть raw TCP-сокет. HTTPS — из-за универсальной совместимости и поддержки прокси.

Альтернативы не подходят. Только UDP для всего — нельзя терять сообщения и нарушать порядок, пришлось бы переизобретать TCP/QUIC. Только TCP для звонков — Head-of-Line Blocking остановит весь медиапоток при потере сегмента. HTTP/REST для сообщений — модель запрос-ответ плохо подходит для постоянного двустороннего потока. gRPC для клиентов — браузеры не поддерживают напрямую. MQTT — ориентирован на IoT-телеметрию, не даёт нужной модели диалогов.

Задание 3. Переход на QUIC/HTTP/3

Преимущества: устранение Head-of-Line Blocking (потеря в одном потоке не блокирует другие — важно при одновременной загрузке файла и приёме сообщений); ускорение установления соединения (1-RTT и 0-RTT); мобильность соединения при смене Wi-Fi на мобильную сеть по Connection ID; встроенное шифрование TLS 1.3; гибкость congestion control.

Ограничения: многие корпоративные и государственные сети блокируют UDP — потребуется fallback на TCP; выше нагрузка на CPU из-за шифрования; сложнее отладка и мониторинг из-за зашифрованных заголовков; MTProto придётся перерабатывать, а не просто менять транспорт; данные в 0-RTT уязвимы к replay-атакам.

Задание 4. Приложение, для которого UDP лучше TCP

Пример — многопользовательская онлайн-игра (шутер, гонки) или VoIP-звонок.

Обоснование: главное — низкая задержка. Позиция игрока обновляется 20–60 раз в секунду; если пакет потерян, ждать повторной передачи бессмысленно — данные устарели. Лучше отбросить пакет и взять следующее состояние. Потеря допустима: пропажа кадра позиции даёт кратковременный «телепорт», сглаживаемый интерполяцией, тогда как Head-of-Line Blocking в TCP вызвал бы «замирание» игры. UDP не требует рукопожатия — клиент шлёт состояние сразу. Критичные события (например, покупка предмета) отправляются отдельно по надёжному каналу TCP/QUIC — типичная гибридная схема.